Qualcomm Technologies, Inc.

Adding a Custom Sensors Algorithm with Sensors Execution Environment (SEE)

80-P9301-67 Rev. C

November 18, 2022

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:35:48 GMT hyunchul.won@lge.com

For additional information or to submit technical questions, go to https://createpoint.qti.qualcomm.com

Confidential – Qualcomm Technologies, Inc. and/or its affiliated companies – May Contain Trade Secrets

NO PUBLIC DISCLOSURE PERMITTED: Please report postings of this document on public servers or websites to DocCtrlAgent@qualcomm.com.

Confidential Distribution: Use or distribution of this item, in whole or in part, is prohibited except as expressly permitted by written agreement(s) and/or terms with Qualcomm Incorporated and/or its subsidiaries.

Not to be used, copied, reproduced, or modified in whole or in part, nor its contents revealed in any manner to others without the express written permission of Qualcomm Technologies, Inc.

All Qualcomm products mentioned herein are products of Qualcomm Technologies, Inc. and/or its subsidiaries.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Other product and brand names may be trademarks or registered trademarks of their respective owners.

This technical data may be subject to U.S. and international export, re-export, or transfer ("export") laws. Diversion contrary to U.S. and international law is strictly prohibited.

Qualcomm Technologies, Inc.
5775 Morehouse Drive
San Diego, CA 92121
U.S.A.

© 2017, 2019, 2021 Qualcomm Technologies, Inc. and/or its subsidiaries. All rights reserved.

Revision history

Revision	Date	Description
A	June 2017	Initial release
B	March 2019	Numerous updates have been made to this document; it should be read in its entirety
C	June 2021	Numerous updates have been made to this document; it should be read in its entirety

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:35:48 GMT
hyungchul.won@lge.com

Contents

Revision history 2

1 Introduction 7

1.1 Conventions 7

1.2 Technical assistance 7

2 SEE overview and sensor APIs 8

2.1 SEE overview 8

2.2 Communication between sensors 9

2.3 Sensor and sensor instance 9

2.4 SEE sensor APIs 10

2.4.1 SEE sensor API – sns_sensor.h 11

2.4.2 SEE sensor API – sns_sensor_instance.h 12

2.4.3 SEE sensor API – sns_register.h 12

2.5 Sensor API messages 13

2.5.1 Protocol buffers and Nanopb 13

2.5.2 SNS .proto files 13

2.5.3 Nanopb-encoded request message 15

2.5.4 Nanopb-encoded event message 15

3 Write a .proto file for customer algorithms 16

3.1 .proto filename 16

3.2 .proto file enum and supported messages 16

3.2.1 Enum and message format 17

3.2.2 Add messages 17

4 Customer algorithm development – OEM1 algorithm 19

4.1 Algorithm source code location 19

4.2 Key functions and files 19

4.2.1 sns_oem1.c 19

4.2.2 sns_oem1_sensor.c 20

4.2.3 sns_oem1_sensor_island.c 22

4.2.4 sns_oem1_sensor_instance.c 22

Qualcomm Confidential - May Contain Trade Secrets
2026-02-27 05:35:48 GMT
hyungchul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4.2.5 sns_oem1_sensor_instance_island.c 23
4.3 Common customizations 25
4.3.1 Resampler vs. direct requests 26
4.3.2 On-change vs. streaming 27
5 Verify OEM1 on target 28
5.1 View OEM1 in USTA 29
5.2 OEM1 test sensor (resident on SLPI) 35
5.3 Debugging logs 36
6 Provide customer algorithms as binary 39
A References 41
A.1 Related documents 41
A.2 Acronyms and terms 41

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:35:48 GMT
hyungchul.won@lge.com

Tables

Table 2-1: SEE high-level components..... 8
Table 2-2: SEE definitions..... 10
Table 4-1: Algo files and descriptions..... 19

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:35:48 GMT
hyungchul.won@lge.com

Figures

Figure 2-1: SEE high-level design..... 8
Figure 2-2: Sensor API..... 11
Figure 2-3: Sensor instance API..... 12

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:35:48 GMT
hyungchul.won@lge.com

1 Introduction

This document describes how to develop and add custom algorithms with the Sensors Execution Environment (SEE), using the OEM1 algorithm as a base.

Readers of this document must be familiar with the SEE architecture. See the following documents:

- *Sensors Execution Environment (SEE) Sensors Deep Dive* (80-P9301-35)
- *Sensors Execution Environment Client API Reference* (80-P9301-36)
- Chipset-specific overview document(s)

1.1 Conventions

Function declarations, function names, type declarations, attributes, and code samples appear in a different font, for example, `cp armcc armcpp`.

Code variables appear in angle brackets, for example, `<number>`.

Commands to be entered appear in a different font, for example, **`copy a:*.* b:`**.

Button and key names appear in bold font, for example, click **Save** or press **Enter**.

1.2 Technical assistance

For assistance or clarification on information in this document, submit a case to Qualcomm Technologies, Inc. (QTI) at https://support.qualcomm.com/.

You will need to register for a Qualcomm ID account and your company must have support enabled to access our Case system. Other systems and support resources are listed on https://qualcomm.com/support. If you need assistance, send email to qualcomm.support@qti.qualcomm.com.

Qualcomm Confidential - May Contain Trade Secrets
2026-02-27 05:35:48 GMT
hyeongchul.won@lge.com

2 SEE overview and sensor APIs

This chapter provides a high-level summary of the SEE framework and relevant APIs. See *Sensors Execution Environment (SEE) Sensors Deep Dive* (80-P9301-35) for details.

2.1 SEE overview

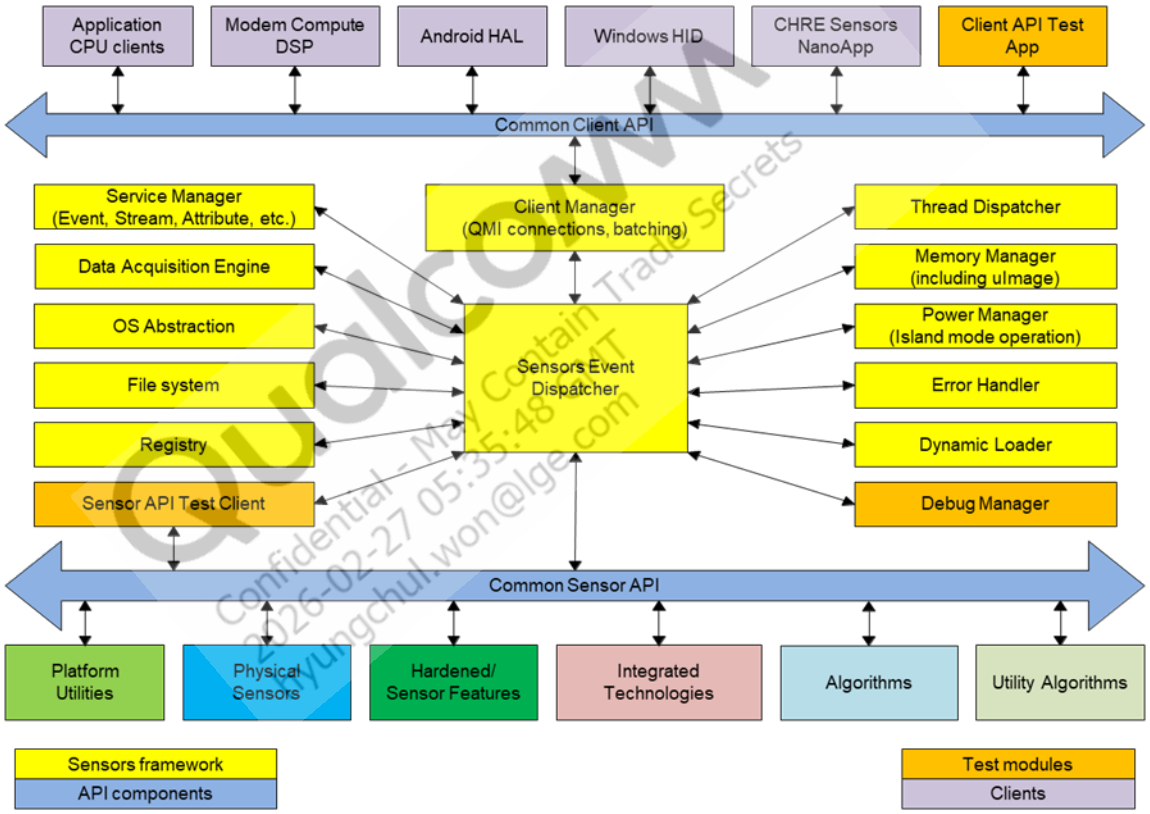


Figure 2-1 SEE high-level design

Table 2-1 SEE high-level components

Component	Definition
API components	▪ Client API – SEE interface with external (to the SSC) clients ▪ Sensor API – Common sensor API for all device drivers
Sensors framework	Core SEE framework that provides services to sensors, manages registry, handles client interface, manages power features, and so on.

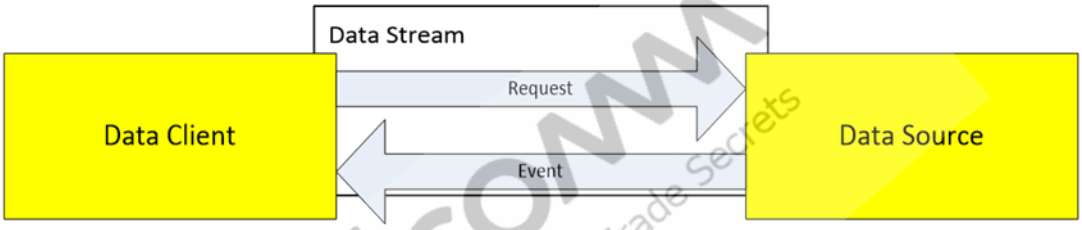
Table 2-1 SEE high-level components (cont.)

Component	Definition
QTI-implemented platform sensors	Modules that provide asynchronous platform feature support
Vendor implemented sensors	Device drivers written by sensor/third-party vendors
Test modules	Test clients to test at each API component
Clients	External clients to the SEE

2.2 Communication between sensors

All communication to, from, and among sensors is performed via request and event messages over data streams.

Message payloads are defined in the protocol buffer format, using the nanopb generator, encoder, and decoder. Message payload length, message ID, and timestamp (in the case of events) are communicated within metadata managed by the SEE framework.



Request messages are sent to enable, disable, and/or reconfigure a sensor. They are always addressed to a specific SUID.

Once the target sensor receives the request message, it sends the request to the sensor instance for proper handling.

Event messages are sent asynchronously by sensor instances to their registered clients, which may be other sensors or sensor instances.

2.3 Sensor and sensor instance

Sensors are producers and/or consumers of asynchronous data. Each sensor can be instantiated one or more times as sensor instances. Each instance operates with a specific configuration. Any request to a sensor for data may result in creation of a sensor instance or sharing of an existing instance.

Sensor instances are created on-demand, as determined by the sensor. Sensors fully manage the lifecycle and configuration of their corresponding instances, and are responsible for sending configuration updates and initial state events to their clients. Vendors are strongly encouraged to serve all client requests with as few instances as possible. A stream of data generated by an instance is sent to all active clients.

Qualcomm Confidential – May Contain Trade Secrets 2026-03-27 05:55:48 GMT hwangchulwon@lge.com

A single sensor instance may be shared and configured by multiple sensors. This mode of operation is typically for a combo driver for hardware sensors, where the sensors represent the supported data types, and the sensor instance is the sole module communicating and configuring the hardware.

Sensor	Sensor instance
▪ An entity that produces a single type of data,such as accelerometer, gyroscope, timer, interrupt, rotation vector, and so on. ▪ SUID – 128-bit number unique to each sensor ▪ Publishes attributes (mandatory and custom) ▪ Manages its instances	▪ An active instance of a sensor that publishes output data events ▪ Sensors may create an instance per client request or share an instance between multiple client requests ▪ Physical sensors typically share a single instance

Table 2-2 SEE definitions

Term	Definition
Sensor	Module that produces and/or consumes asynchronous data
Sensor instance	Instantiation of a sensor that runs at a specific configuration
Sensor unique identifier (SUID)	Unique 128-bit ID per sensor
Service	Module that provides a synchronous interface for common utilities
Data stream	Unique connection between a client and data source
Request	▪ A configuration message sent by a client to a sensor ▪ See sns_request.h
Event	▪ Asynchronous output data message generated by a sensor instance ▪ See sns_sensor_event.h

2.4 SEE sensor APIs

Common APIs include:

- Physical sensors (device drivers)
- Virtual sensors (algorithms)

APIs are defined in the following files:

- `\slpi_proc\ssc\inc\sns_sensor.h`
- `\slpi_proc\ssc\inc\sns_sensor_instance.h`
- `\slpi_proc\ssc\inc\sns_register.h`

NOTE Though this document refers to "slpi_proc" for SLPI for SM8xxx tier chipsets, this document is applicable to ADSP LPI targets as well for other tiers, and “modem_proc” for IOT chipsets where SEE is supported.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:35:48 GMT kyunchul.won@lge.com

2.4.1 SEE sensor API – sns_sensor.h

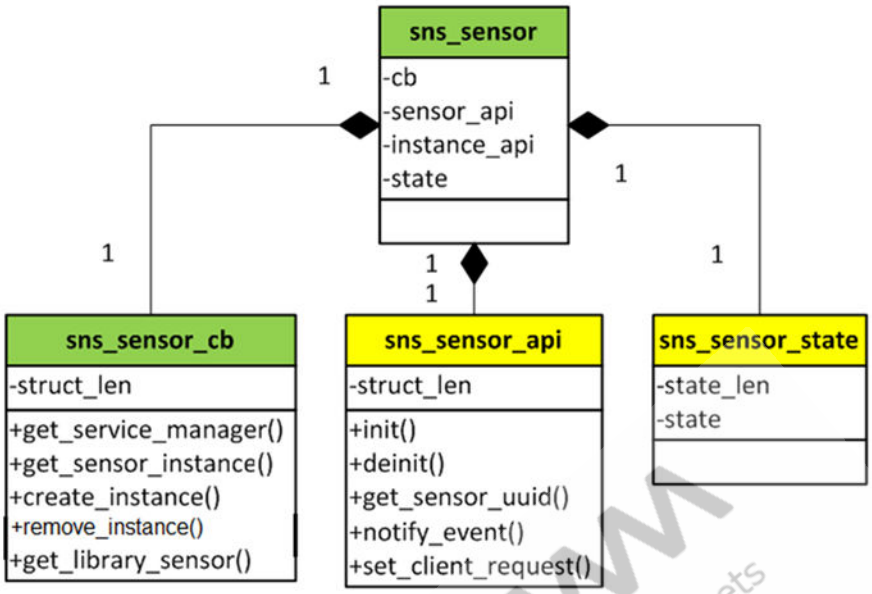


Figure 2-2 Sensor API

`sns_sensor struct` defines the following:

- `sns_sensor_cb` – Callbacks in the SEE framework available to a sensor
  - Gets the service manager handle (`.get_service_manager()`)
  - Gets the next instance for the sensor (`.get_sensor_instance()`)
  - Creates a new instance (`.create_instance()`)
  - Removes an existing instance (`.remove_instance()`)
  - Gets the next sensor supported in this sensor's library (`.get_library_sensor()`)
- `sns_sensor_api` – Functions that every sensor must implement and are called only by the SEE framework
  - Initializes a sensor (`.init()`) – Called during the sensor registration
  - Destroys a sensor (`.deinit()`) – Called when the sensor reports any failure error code
  - Gets the unique sensor identifier (`.get_sensor_uid()`)
  - Notifies a sensor that an event is available from a dependent sensor (`.notify_event()`)
  - Updates a client request for a sensor (`.set_client_request()`)
- `sns_sensor_state` – Private state maintained by a sensor
  - Memory for the sensor state is allocated by the SEE framework during the sensor registration

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 05:35:19 GMT hyunchul.won@lge.com

2.4.2 SEE sensor API – sns_sensor_instance.h

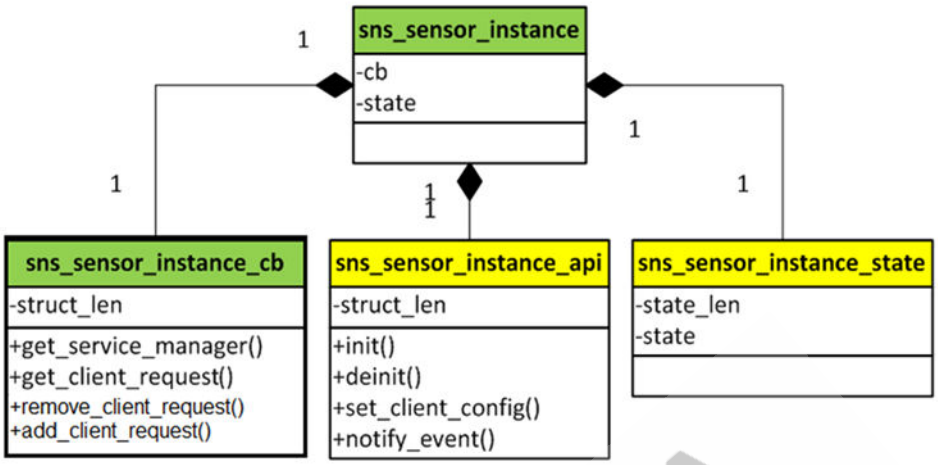


Figure 2-3 Sensor instance API

`sns_sensor_instance` struct defines the following:

- `sns_sensor_instance_cb` – Callbacks in the SEE framework available to a sensor instance
  - Gets the service manager handle (`.get_service_manager()`)
  - Gets the next client request associated with an instance (`.get_client_request()`)
  - Removes a client request handled by an instance (`.remove_client_request()`)
  - Adds a client request handled by an instance (`.add_client_request`)
- `sns_sensor_instance_state` – Private state maintained by a sensor instance
  - SEE framework allocates the memory for a sensor instance state during the creation of a sensor instance

`sns_sensor_instance_api` defines functions that every sensor instance must implement and are called by its sensor or the SEE framework. It does the following:

- Initializes a sensor instance (`.init()`) – Called by the SEE framework when an instance is created
- Destroys a sensor instance (`.deinit()`) – Called by the SEE framework when its sensor requests a `remove_instance()`
- Sets instance configuration (`.set_client_config()`) – Called by the sensor to set the instance configuration
- Notifies an instance that an event is available from a dependent sensor (`.notify_event()`) – Called by the SEE framework

2.4.3 SEE sensor API – sns_register.h

Each driver library implements a registration function of type `sns_register_sensors()`.

`sns_register_cb` struct defines callback to initialize a sensor (`.init_sensor()`).

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

This function calls `init_sensor()` on each supported sensor within the library.

2.5 Sensor API messages

2.5.1 Protocol buffers and Nanopb

Google protocol buffers are a data format for serializing data structures into a byte stream agnostic of language and platform. Data structure information is defined in the form of a protocol buffer message in a .proto file. The .proto file is run through a protocol buffer compiler to generate data structures that can be used programmatically.

The protocol buffer design also provides special code to encode (serialize at the source)/ decode (deserialize at the sink) data to and from byte streams.

See https://developers.google.com/protocol-buffers/ for detailed information about Google protocol buffers.

Nanopb – C implementation of Google protocol buffers

- Small code size
- Supports 32-bit processor architecture
- See https://jpa.kapsi.fi/nanopb/ for detailed information about nanopb

SEE uses nanopb protocol buffers for the following:

- All request and event messages exchanged between sensors
  - A sensor/instance must encode the payload (if present) for all requests it sends to its dependents.
  - A sensor/instance must decode the payload (if present) for all requests it receives.
  - A sensor/instance must encode the payload (if present) for all events it publishes.
  - A sensor/instance must decode the payload (if present) for all events it receives from its dependents.
  - Some requests/events may not have a message body. In this case, decoding/encoding the payload is not expected. Such messages are typically processed by their message ID.
- Representing attribute data – All attribute values are in nanopb-encoded format.
- Diag log packet payload – All payloads in diag log packets are in nanopb-encoded format.

2.5.2 SNS .proto files

.proto files contain the following API elements:

- Protocol buffer message definitions
- Documentation

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 05:35:48 GMT hyunchul.yoon@lge.com

Standard message definitions are in `\slpi_proc\ssc_api\pb\sns_std_*.proto`.

- `sns_std.proto` contains the following standard definitions:
  - Framework-defined message ID
  - Standard request message
  - Batching spec
  - Attribute request and event
  - Error event message
- `sns_std_sensor.proto` contains the following standard definitions:
  - Message IDs for request and event API for standard sensors
  - Streaming request and event messages
  - Sensor sample status types
  - Common attribute types
  - Standard attribute IDs
  - Physical sensor configuration event message

Standard message definitions are in `\slpi_proc\ssc_api\pb\sns_std_*.proto`.

- `sns_std_type.proto` contains common API-type definitions, for example:
  - SUID message
  - Attribute event and value message
  - Common error types
- sns_std_event_gated_sensor.proto contains the API for event gated sensors:
  - Config message ID
  - API documentation

Physical sensor-specific API definitions and documentation are present in sensor specific .proto files, for example, `sns_accel.proto`, `sns_proximity.proto`, `sns_motion_detect.proto`, and so on.

Platform sensor API definitions and documentation are present at `\slpi_proc\ssc_api \pb\`, for example, `sns_timer.proto`, `sns_interrupt.proto`, and `sns_async_com_port.proto`.

A framework-related API for SUID, registry, diag, and data acquisition engine (DAE) is defined in the following:

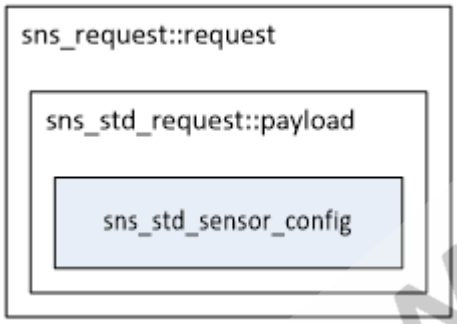
- `sns_suid.proto`
- `sns_registry.proto`
- `sns_diag.proto`
- `sns_dae.proto`

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 05:35:48 GMT hyungshul.won@lge.com

2.5.3 Nanopb-encoded request message

`sns_request::request` contains the following:

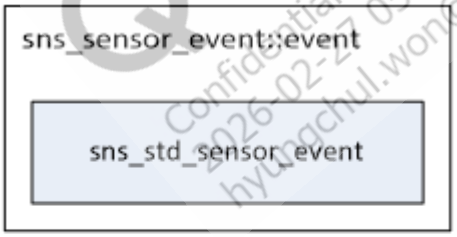
- Nanopb-encoded `sns_std_request` message
  - `sns_std_request` payload
    - Typically contains nanopb-encoded `sns_std_sensor_config` message
    - Any other sensor-specific request message
    - May be NULL for empty request payloads

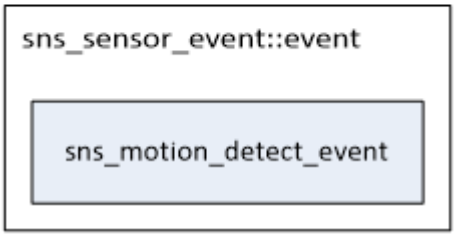


2.5.4 Nanopb-encoded event message

`sns_sensor_event::event` contains the following:

- Nanopb-encoded event message; typically, the `sns_std_sensor_event` message but may be different based on the sensor. For example, for `motion_detect`, it is `sns_motion_detect_event`.





Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 05:35:15 GMT hyungchul.won@lge.com

3 Write a .proto file for customer algorithms

The .proto file defines the messages used by the custom algorithm, using the protocol buffer specification language.

3.1 .proto filename

The proto file name for an algorithm should be the same as the attribute `SNS_STD_SENSOR_ATTRID_API` value set in the respective `sns_<algo_name>_sensor.c file`.

Example code in `sns_oem1_sensor.c`:

```
{
    sns_std_attr_value_data values[] = {SNS_ATTR, SNS_ATTR};
    char const proto1[] = "sns_oem1.proto";
    char const proto2[] = "sns_std_sensor.proto";
    values[0].str.funcs.encode = pb_encode_string_cb;
    values[0].str.arg = &((pb_buffer_arg)
        { .buf = proto1, .buf_len = sizeof(proto1) });
    values[1].str.funcs.encode = pb_encode_string_cb;
    values[1].str.arg = &((pb_buffer_arg)
        { .buf = proto2, .buf_len = sizeof(proto2) });
    sns_publish_attribute(this, SNS_STD_SENSOR_ATTRID_API,
        values, ARR_SIZE(values), false);
  }
```

.proto filename for this algorithm should be `sns_oem1.proto`.

3.2 .proto file enum and supported messages

Each supported message has a message ID associated with it. All supported message IDs are listed in an enum in the .proto file.

Custom algorithms can take advantage of standard messages defined in `slpi_proc\ssc_api\pb\sns_std_sensor.proto` by importing the proto file.

```
import "sns_std_sensor.proto";
```

Refer to `slpi_proc\ssc_api\pb\sns_oem1.proto`.

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 05:35:16 GMT hyung-shul.wang@qti.com

3.2.1 Enum and message format

The enum name format should be `<filename>_msgid` (filename does not contain the .proto extension) and the supported message format within the enum should be `<enum_name>_<message_name> = <message_ID>`.

For example, `SNS_OEM1_MSGID_SNS_OEM1_DATA` is the message ID used for the output data event generated by the OEM1 sensor.

```
enum sns_oem1_msgid
{
  option (nanopb_enumopt).long_names = false;

  SNS_OEM1_MSGID_SNS_OEM1_DATA = 1024;
}
```

For a message ID added in the enum, there is a corresponding protobuf message which contains fields based on the purpose of the message ID.

For example, `SNS_OEM1_MSGID_SNS_OEM1_DATA` uses `sns_oem1_data` for the OEM1 sensor event data which contains values along x, y, z axis.

```
// Data Message
// Output data event generated by the oem1 sensor.
message sns_oem1_data
{
  // oem1 Vector along axis x,y,z in m/s2
  repeated float oem1 = 1 [(nanopb).max_count = 3];
  // Accuracy of the data
  required sns_std_sensor_sample_status accuracy = 2;
```

3.2.2 Add messages

Additional custom messages within the scope of the algorithm can also be written, named, and used in accordance to the above structure as per user convenience.

Example

```
import "sns_std_sensor.proto";

enum sns_oem1_msgid
{
  option (nanopb_enumopt).long_names = false;

  // Uses message: sns_oem1_config
  // Purpose:
  //   1. A stream request from a client to the oem1 sensor.
  //   2. Add more parameters as compared to the sns_std_sensor_config
  //   3. A config/ack event from a sensor to the client.
  SNS_OEM1_MSGID_SNS_OEM1_CONFIG = 520;
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:35:48 GMT hyungchul.yoon@lge.com

```
  SNS_OEM1_MSGID_SNS_OEM1_DATA = 1024;
}

// Configuration Message
// Input Configurable data to the oem1 sensor
message sns_oem1_config
{
  // Sample rate in Hz
  required float sample_rate = 1;
  optional float ip_param1 = 2;
  optional int32 ip_param2 = 3;
  optional fixed32 ip_param3 = 4;
}
```

OEM1 supports `SNS_STD_SENSOR_MSGID_SNS_STD_SENSOR_CONFIG (513)` and `SNS_STD_SENSOR_MSGID_SNS_STD_ON_CHANGE_CONFIG (514)` which are defined in the `sns_std_sensor.proto`.

`SNS_OEM1_MSGID_SNS_OEM1_CONFIG (520)` is added to the enum to support a config message with additional parameters.

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:35:48 GMT
hyungchul.won@lge.com

4 Customer algorithm development – OEM1 algorithm

OEM1 is a sample algorithm on top of the SEE framework that QTI provides to check code. It streams accel sensor data.

4.1 Algorithm source code location

Like other sensors, algorithms reside in the `slpi_proc\ssc\sensors\<algo_name>` directory with SEE.

OEM1 is provided as a sample algorithm in the default QTI SLPI code in the following location:

`slpi_proc\ssc\sensors\oem1\`

4.2 Key functions and files

Table 4-1 Algo files and descriptions

Files	Description
`sns_<algo_name>.c`	Function to register algorithm library
`sns_<algo_name>_sensor.c` `sns_<algo_name>_sensor.h`	▪ Normal mode APIs for algorithm ▪ Data types for algorithm
`sns_<algo_name>_sensor_island.c`	Island mode functions for algorithm
`sns_<algo_name>_sensor_instance.c` `sns_<algo_name>_sensor_instance.h`	▪ Normal mode functions for the sensor instance ▪ Sensor instance data types for the algorithm
`sns_<algo_name>_sensor_instance_island.c`	Island mode functions for the algorithm instance

4.2.1 sns_oem1.c

- `sns_oem1_register` function is used to register the sensor.

  After the build, you can see all the sensors that are registered in the `sns_static_sensors.c` file. (For all sensors that are up and running, you should see the corresponding registry functions in this file (`sns_static_sensors.c`).)

```
sns_rc sns_oem1_register(sns_register_cb const *register_api)
{
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:35:48 GMT hyungchul.won@lge.com

```
  register_api->init_sensor(sizeof(sns_oem1_sensor_state),
                            &sns_oem1_api,
                            &sns_oem1_sensor_instance_api);
  return SNS_RC_SUCCESS;
}
```

- `sns_register_cb` is a structure that contains callback functions in the framework (such as `init_sensor` is a callback function).
- `init_sensor` is used to allocate and initialize a sensor; see SEE sensor API – sns_register.h for more details.

4.2.2 sns_oem1_sensor.c

- `sns_oem1_init` – Function used to initialize and initiate the lookup data and to publish sensor attributes. These attributes are published from sns_oem1_init through the function publish_attributes function.

  Standard attributes of the sensor are seen in the file `sns_std_sensor.pb.h` file (autogenerated from the `sns_std_sensor.proto` file using Nanopb).

  Examples: `SNS_STD_SENSOR_ATTRID_NAME`, `SNS_STD_SENSOR_ATTRID_TYPE`,`SNS_STD_SENSOR_ATTRID_VENDOR`, `SNS_STD_SENSOR_ATTRID_VENDOR`, and so on.

  `SNS_SUID_LOOKUP_INIT` and `sns_suid_lookup_add` are also performed from the `sns_oem_init` function. See `sns_suid_util.h` for details.

  - `SNS_SUID_LOOKUP_INIT` – Initializes the data created by `SNS_SUID_LOOKUP_DATA`

    Example: `SNS_SUID_LOOKUP_INIT(state > suid_lookup_data, NULL);`

  - `sns_suid_lookup_add` – Initiates SUID lookup for the default sensor of a data type

    Example: `sns_suid_lookup_add(this, &state > suid_lookup_data, "accel");`

    Looks up all dependency sensors of OEM1 (such as registry, resampler).

- `sns_oem1_notify_event` – Notifies client that some data has been received

  If `suid_lookup` is completed, that is, all dependent sensors are available, then publish the sensor through the function `publish_available` and perform `sns_suid_lookup_deinit`.

  Example:

```
  if(sns_suid_lookup_complete(&state->suid_lookup_data))
  {
#ifdef OEM1_SUPPORT_REGISTRY
    if(state->first_pass == true){
      state->first_pass = false;
      sns_oem1_registry_req(this);
    }
#endif
    publish_available(this);
    sns_suid_lookup_deinit(this, &state->suid_lookup_data);
  }
```

Qualcomm Confidential – May Contain Trade Secrets 2020-02-27 05:25:49 GMT hwkrchul.won@lge.com

```
sns_suid_lookup_deinit - Deinitialize the data initialized by
SNS_SUID_LOOKUP_INIT

static void
publish_available(sns_sensor *const this)
{
  sns_std_attr_value_data value = sns_std_attr_value_data_init_default;
  value.has_boolean = true;
  value.boolean = true;
  sns_publish_attribute(
      this, SNS_STD_SENSOR_ATTRID_AVAILABLE, &value, 1, true);
}
```

- `sns_oem1_set_client_request` – Adds, removes, or updates a client's request to this sensor.

  Find the instance (if any) that is presently servicing this request and update in curr_inst (current instance).

  (`sns_sensor_util_find_instance` is a utility function to find the instance that is presently servicing this request.)

  - If remove is set, remove the current request.

    Example: `curr_inst > cb > remove_client_request(curr_inst, curr_req);`

  - If `message_id` in `new_req` (new request) is `SNS_STD_SENSOR_MSGID_SNS_STD_SENSOR_CONFIG` (streaming) (or) `SNS_STD_SENSOR_MSGID_SNS_STD_ON_CHANGE_CONFIG` (on-change):

    i. Decode the new request using `pb_decode` (not required for on-change).

    - New request might contain sample rates and any custom parameters.
    - For on-change, a new request will not contain any data, so decoding is not required.

    ii. Create a new instance of the sensor (allocate and initialize a new sensor instance to be associated with this sensor).

    Example: `this > cb > create_instance(this, (sizeof(sns_oem1_inst_state)));`

    iii. `add_client_request` (assign this sensor instance to service the client request)

    `add_client_request(rv_inst, new_req)`

    iv. `set_client_config` (update a sensor instance configuration to this sensor request)

    `set_client_config(rv_inst, &decoded_req)`

    v. `remove_instance` (if current instance is not NULL and get_client_request is NULL)

    `remove_instance(curr_inst);`

Qualcomm Confidential – May Contain Trade Secrets 2026-03-27 05:35:18 GMT jyangchun@lge.com

4.2.3 sns_oem1_sensor_island.c

The OEM1 sensors API is defined in this file:

```
{
    .struct_len = sizeof(sns_sensor_api),
    .init = &sns_oem1_init,
    .deinit = &sns_oem1_deinit,
    .get_sensor_uid = &sns_oem1_get_sensor_uid,
    .set_client_request = &sns_oem1_set_client_request,
    .notify_event = &sns_oem1_notify_event,
};
```

See SEE sensor API – sns_sensor.h for details on sns_sensor_api functions.

- `sns_oem1_get_sensor_uid` – Gets the unique sensor identifier for OEM1.

4.2.4 sns_oem1_sensor_instance.c

- sns_oem1_inst_init – Initializes a sensor instance to its default state:
  a. Get the sensor SUID's and services and fill the structure `sns_oem1_inst_state` appropriately.
  b. Create `accel_stream` if necessary (create connection with accel steam if direct accel steam macro is enabled).
  c. Copy platform-specific configuration `sns_oem1_sensor_state` to `sns_oem1_inst_state`.
- `sns_oem1_inst_deinit` – Removes all streams that were created either in `sns_oem1_inst_init` or `sns_oem1_inst_set_client_config` (release all hardware and software resources associated with this sensor instance)
- `sns_oem1_inst_set_client_config` – Updates a sensor instance configuration to this sensorRequest.

  If `message_id` is `SNS_STD_SENSOR_MSGID_SNS_STD_ON_CHANGE_CONFIG` or `SNS_STD_SENSOR_MSGID_SNS_STD_SENSOR_CONFIG`:
  a. Initialize state (update `client_config` and `sample_rate` from the client request).
  b. If direct accel request (no resampler), then `send_request` to accel sensor.

```
sns_std_sensor_config accel_config = {.sample_rate = state->config.sample_rate};
encoded_len = pb_encode_request(buffer,
    sizeof(buffer),
    &accel_config,
```

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 05:35:48 GMT hyunchul.wan@qc.com

```
sns_std_sensor_config_fields,
NULL);
```

- For details on resampler vs. direct requests, see Resampler vs. direct requests.
- Create a connection with accel sensor (done in `sns_oem1_inst_init`).

  `accel_stream` (stream reference allocated by the framework is updated).
- `send_request` – Send a request to another service/sensor (`sns_data_stream.h`).

  Example: `state > accel_stream > api > send_request(state > accel_stream, &request);`

c. If resampler is enabled (OEM1 is getting accel data through resampler):

```
sns_resampler_config resampler_config =
sns_resampler_config_init_default;
sns_memscpy(&resampler_config.sensor_uid,
     sizeof(resampler_config.sensor_uid),
     &state->accel_suid,
     sizeof(state->accel_suid));
     resampler_config.resampled_rate = state->config.sample_rate;
     resampler_config.rate_type = SNS_RESAMPLER_RATE_MINIMUM;
     resampler_config.filter = false;
     encoded_len = pb_encode_request(buffer, sizeof(buffer),
      &resampler_config, sns_resampler_config_fields, NULL);
```

- Create a connection with resampler sensor.

  `resampler_accel_stream` (stream reference allocated by the framework is updated)
- `send_request` – Send a request to another service/sensor (`sns_data_stream.h`).

  Example: `state > resampler_accel_stream > api > send_request(state > resampler_accel_stream, &request);`

  Request is sent on data stream `resampler_accel_stream`. Framework will copy the request.

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 05:35:48 GMT hyunchul.won@lge.com

4.2.5 sns_oem1_sensor_instance_island.c

The OEM1 sensor instance API is defined in this file:

```
sns_sensor_instance_api sns_oem1_sensor_instance_api =
{
    .struct_len = sizeof(sns_sensor_instance_api),
    .init = &sns_oem1_inst_init,
    .deinit = &sns_oem1_inst_deinit,
    .set_client_config = &sns_oem1_inst_set_client_config,
    .notify_event = &sns_oem1_inst_notify_event
};
```

See SEE sensor API – sns_sensor_instance.h for details on sns_sensor_instance_api functions.

- `sns_oem1_inst_notify_event` – Notifies the client that some data has been received. (Processes incoming events to the OEM1 sensor instance.)

  a. Decode and print the accel data.

  b. If resampler is not enabled, get the event from `accel_stream`:

     Example: `accel_event_in = state > accel_stream > api > peek_input(state > accel_stream);`

     If `(accel_event_in = state > accel_stream > api > peek_input(state > accel_stream);`:

     – Decode the event (you will get the accel data).

     – OEM1 algorithm logic:

```
//This is dummy logic for OEM1 demonstration purposes
//OEMs can replace with their algo logic
if(0 < temp[2])
{
  current_state = OEM1_FACING_UP;
  accel_payload[0]=100;
  accel_payload[1]=temp[1];
  accel_payload[2]=temp[2];
}
else
{
  current_state = OEM1_FACING_DOWN;
  accel_payload[0]=state->down_value;
  accel_payload[1]=temp[1];
  accel_payload[2]=temp[2];
}
```

     `temp[0-2]` contains input to the algorithm from the dependent sensor, that is, x, y, and z axes of accelerometer data in the default implementation.

     `accel_payload[0-2]` contains the output of the algorithm generated by OEM1.

     – Encode and publish an outgoing `sns_std_sensor_event` event through `pb_send_sensor_stream_event`.

  c. If resampler is enabled, get the data from `resampler_stream`:

     Example: `state > resampler_stream > api > peek_input(state > resampler_stream)`

     If `(SNS_STD_SENSOR_MSGID_SNS_STD_SENSOR_EVENT == resampler_event_in >message_id)`:

     – Decode the event (you will get the accel data).

     – OEM1 algorithm logic:

```
//This is dummy logic for OEM1 demonstration purposes
//OEMs can replace with their algo logic
if(0 < data[2])
```

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 05:25:18 GMT hwung-hsu.wan@lge.com

```
{
  current_state = OEM1_FACING_UP;
  oem1_payload[0]=100;
  oem1_payload[1]=data[1];
  oem1_payload[2]=data[2];
}
else
{
  current_state = OEM1_FACING_DOWN;
  oem1_payload[0]= state->down_value;
  oem1_payload[1]=data[1];
  oem1_payload[2]=data[2];
}
```

`data[0-2]` contains input to the algorithm from the dependent sensor.

`oem1_payload[0-2]` contains the output of the algorithm generated by OEM1.

– Encode and publish an outgoing `sns_std_sensor_event event` through `pb_send_sensor_stream_event`.

4.3 Common customizations

OEM1 is designed to highlight various common customizations that the SEE algorithm developer may need, for example:

- Supporting output as streaming or on-change
- Algorithm requiring sensor input at a fixed rate
- Registry support (reading certain configuration parameters)

Custom flag (in sns_oem1_sensor.h)	Description
`#define OEM1_SUPPORT_DIRECT_SENSOR_REQUEST`	▪ With this flag enabled: ▫ OEM1 algorithm requests the data from accel sensor directly. ▫ The rate at which algorithm receives input sensor samples (accel) is dependent on ODR of accel driver, for example, if there is a concurrent client for accel sensor at a higher rate, algorithm would receive accel data at higher rate). ▪ With this flag disabled: ▫ OEM1 requests accel data via QTI-provided resampler module. ▫ Resampler module always confirms the rate is same as requested and it is oblivious to any other concurrent requests.
`#define OEM1_SUPPORT_EVENT_TYPE`	▪ With this flag enabled: ▫ The algorithm is set to have on-change/event output. ▫ The OEM1 algorithm reports the data only if output changes (and not every output).

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:35:48 GMT hyungchul.yoon@lge.com

Custom flag (in sns_oem1_sensor.h)	Description
	▪ With this flag disabled: ▫ OEM1 output is set to streaming type. ▫ Every time the algorithm receives input from accel, it will calculate and generate event.
`#define OEM1_SUPPORT_REGISTRY`	▪ This flag can be enabled if the requirement is for the algorithm to read certain configuration parameters from registry. ▪ Besides enabling OEM1_SUPPORT_REGISTRY on the SLPI side, the sns_oem1.json file should also be placed on the AP side where the rest of the JSON files are present. `adb push sns_oem1.json /vendor/etc/sensors/config`

OEM1 also depends on the following sensors:

- accel
- registry
- resampler – If enabled

To add any other dependency, such as streaming gyro (or) mag along with accel, change to the appropriate sns_oem1_sensor_state data structure.

Example: `SNS_SUID_LOOKUP_DATA(3) suid_lookup_data;` – The number 3 represents the number of dependencies.

For more details on these customizations and to understand associated changes required in the algorithm, the SEE algorithm developer should study the code around the abovementioned flags, have a thorough understanding of the OEM1 template code, and map it to the requirements of the algorithm.

4.3.1 Resampler vs. direct requests

Clients/sensors that require a fixed rate of dependent sensor will typically register with the resampler. By default, OEM1 is shipped with resampler enabled, for example, `OEM1_SUPPORT_DIRECT_SENSOR_REQUEST` disabled.

`SNS_RESAMPLER_MSGID_SNS_RESAMPLER_CONFIG` is handled for resampler requests vs. `SNS_STD_SENSOR_MSGID_SNS_STD_SENSOR_CONFIG` for direct request.

- Sample `send_request` with resampler config and with direct sensor config:
  - Resampler Configuration Message is present in `sns_resampler_config structure` (`sns_resampler.proto`)

```
sns_request request = (sns_request){
  .message_id = SNS_RESAMPLER_MSGID_SNS_RESAMPLER_CONFIG,
  .request_len = encoded_len, .request = buffer };
  state->resampler_stream->api->send_request(state->resampler_stream,
&request);
    vs.
  sns_request request = (sns_request){
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
.message_id = SNS_STD_SENSOR_MSGID_SNS_STD_SENSOR_CONFIG,
.request_len = encoded_len, .request = buffer };
state->accel_stream->api->send_request(state>accel_stream, &request);
```

4.3.2 On-change vs. streaming

For on-change, the request from client is empty, so there is no need to decode the request. By default, OEM1 is shipped with output set to on-change, for example, `OEM1_SUPPORT_EVENT_TYPE` enabled.

`SNS_STD_SENSOR_MSGID_SNS_STD_ON_CHANGE_CONFIG` is handled for on change vs. `SNS_STD_SENSOR_MSGID_SNS_STD_SENSOR_CONFIG` for streaming.

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:35:48 GMT
hyungchul.won@lge.com

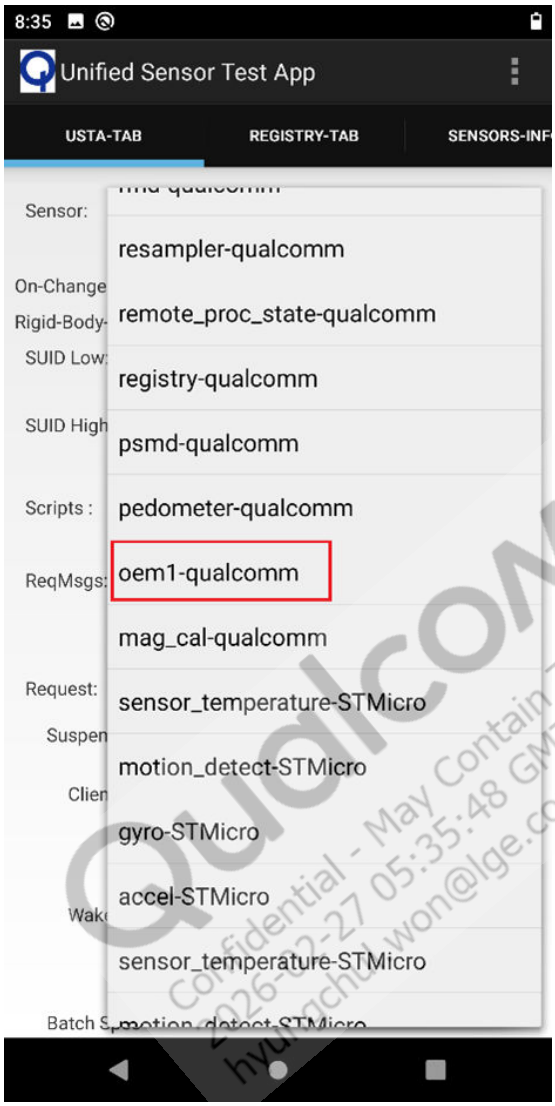
5 Verify OEM1 on target

Unified Sensor Test Application (USTA) is the recommended method for testing OEM1. SLPI resident test sensor can also be used for OEM1 testing, but this is only recommended as an intermediate step if USTA is not available for some reason on a particular platform.

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:35:48 GMT
hyungchul.won@lge.com

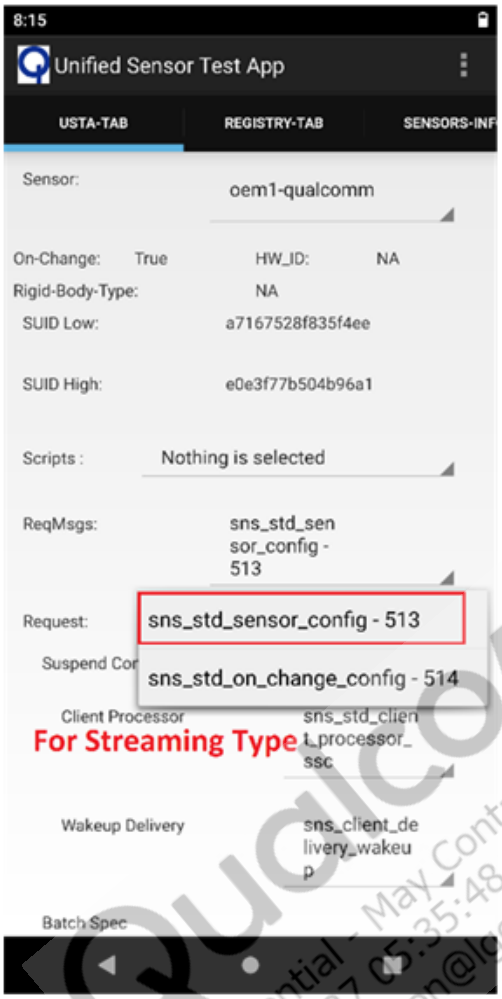
5.1 View OEM1 in USTA

1. Open USTA and select **oem1-qualcomm** in the Sensor drop-down.



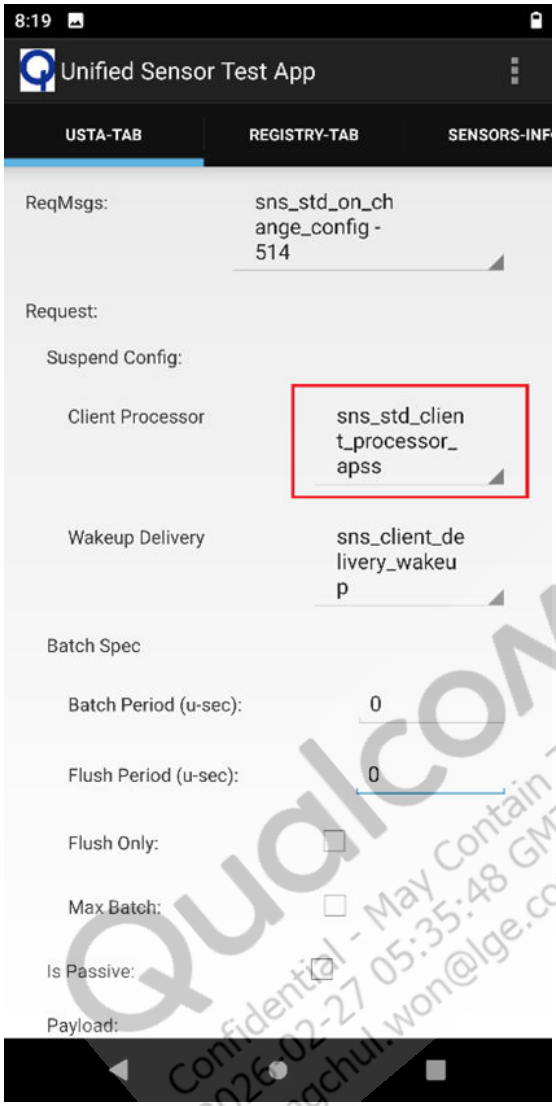
Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:35:48 GMT hyunchol.won@lge.com

2. In the ReqMsgs drop-down, select **sns_std_sensor_config – 513** for streaming and **sns_std_on_change_config – 514** for on-change type.



Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:35:48 GMT hyungchul.won@lge.com

3. In client processor, select **sns_std_client_processor_apss** if streaming to an application processor.

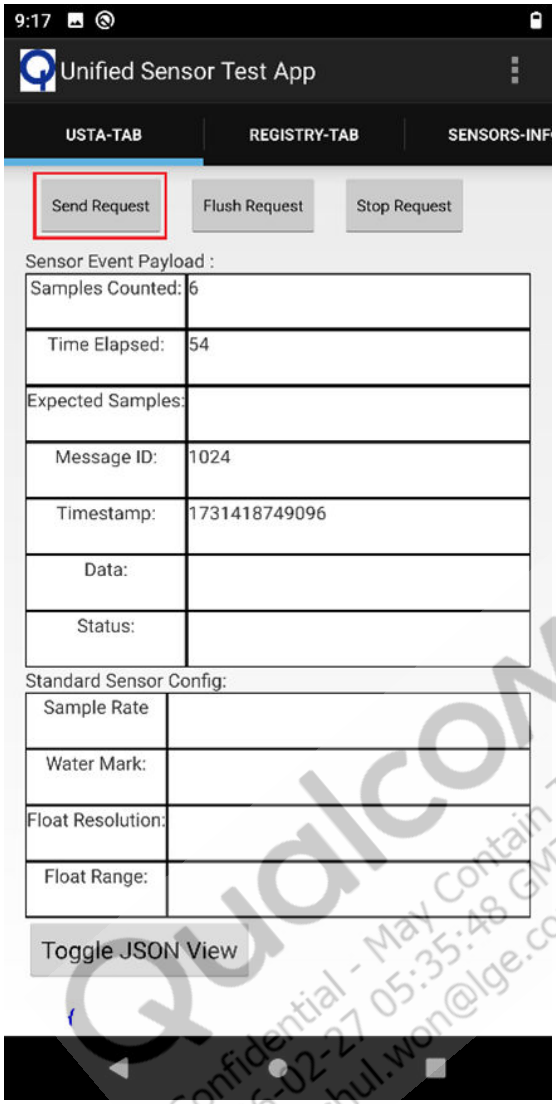


Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:35:48 GMT hyungchul.won@lge.com

4. In the **Batch Period** and **Flush Period** fields, type `0`. In the **sample_rate(Hz)** field (streaming type only), type `1`. (These values can be changed.)

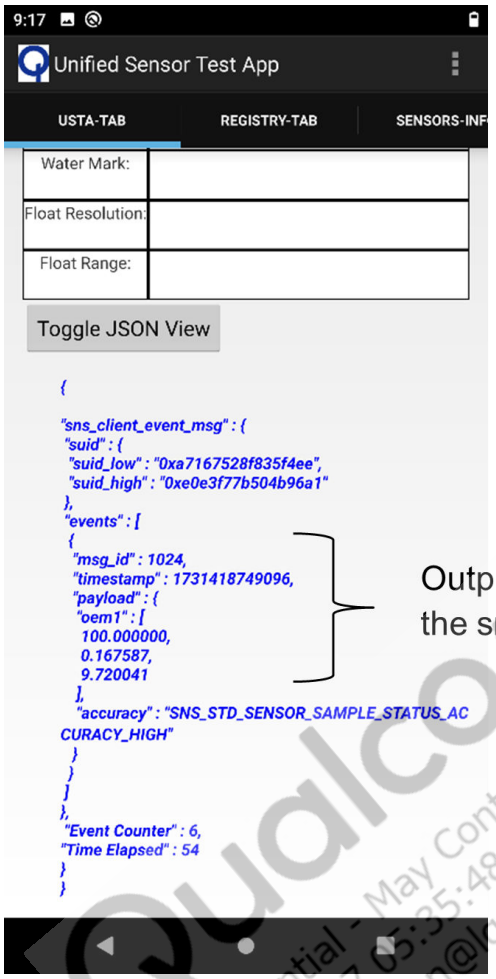
Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:35:48 GMT
hyungchul.won@lge.com

5. Click **Send Request**. Samples Counted, Time Elapsed, and so on, appear on the screen.



Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:35:48 GMT
hyungchul.won@lge.com

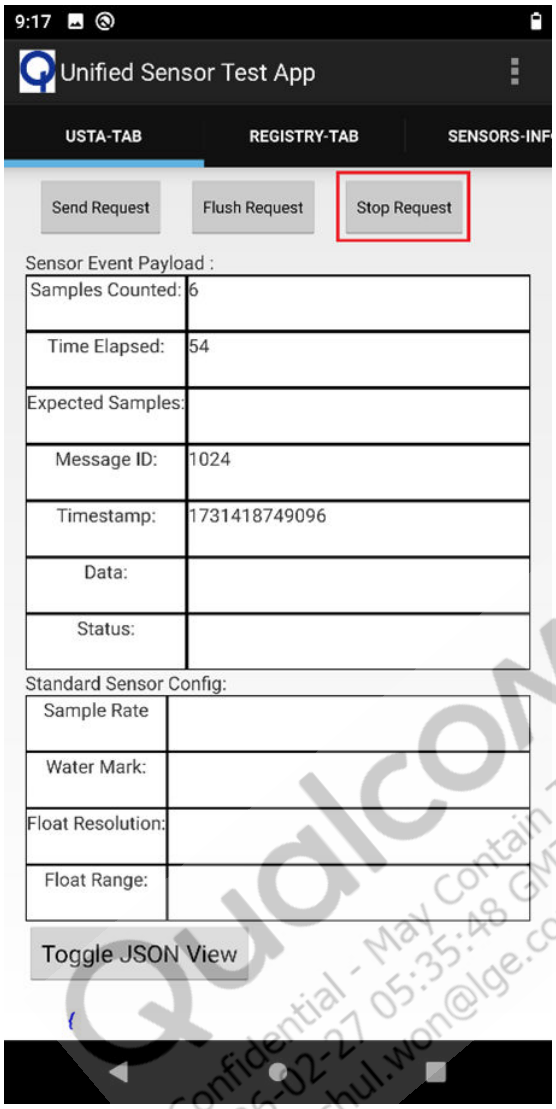
6. Toggle the JSON View to view the output from the OEM1 algorithm as defined in Key Functions. For more information, see sns_oem1_sensor_instance_island.c.



Output from OEM1 algorithm as defined in the sns_oem1_inst_notify_event

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:35:48 GMT
hyungchul.won@lge.com

7. Click **Stop Request** to stop the sensor stream.



Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:35:48 GMT
hyungchul.won@lge.com

5.2 OEM1 test sensor (resident on SLPI)

The test sensor located in `slpi_proc\ssc\sensors\test` supports OEM1 algorithm testing from the test sensor. OEMs can similarly add files for custom algorithms.

Refer to following files:

- `slpi_proc\ssc\sensors\test\src\sns_test_oem1.c`
- `slpi_proc\ssc\sensors\test\src\sns_test_sensor.c`
- `slpi_proc\ssc\sensors\test\inc\sns_test_oem1.h`
- `slpi_proc\ssc\sensors\test\build\sns_test.scons`

Use the following build flags in the build command:

- `SNS_DELAY_INIT` – To view the bootup time logs and OEM1 initialization logs
- `SNS_TEST_OEM1` – To enable test sensor to stream OEM1

NOTE To reiterate, USTA is the recommended method to validate OEM1 or any custom algorithm.

5.3 Debugging logs

Open QXDM Professional™ (QXDM Pro). On the filtered view window, right-click and select **[Configure…]** > **[Message Packets]** > **Click SNS**. This sets the sensors log mask.

Logs will appear in QXDM Pro for a client request sent for OEM1 sensor (see View OEM1 in USTA).

The following are sample QXDM logs for OEM1 as on-change type with resampler enabled:

```
[sns_stream_service.c  148] Created data stream to Sensor 'oem1' (b25628a8)
from Instance 'cm' (b2021328): b2022268
[   sns_oem1_sensor.c  393] sns_oem1_set_client_request
[sns_oem1_sensor_instance.c  100] sns_oem1_inst_set_client_config
[sns_oem1_sensor_instance.c  154] Enable resampler at rate 0
[sns_oem1_sensor_instance.c  172] Before create_sensor_instance_stream
[sns_stream_service.c  148] Created data stream to Sensor 'resampler'
(b201ab68) from Instance 'oem1' (b255c148): b2567468
[sns_oem1_sensor_instance.c  179] After create_sensor_instance_stream
[sns_oem1_sensor_instance.c  183] resampler_stream NOT NULL
[sns_oem1_sensor_instance.c  192] oem1 sent accel request to resampler at
rate 0
[sns_oem1_sensor_instance_island.c   49] sns_oem1_inst_process_resampler_event
[sns_oem1_sensor_instance_island.c   39] OEM1-1, timestamp-0x15ce59f7c6
[sns_oem1_sensor_instance_island.c  150] Received
SNS_RESAMPLER_MSGID_SNS_RESAMPLER_CONFIG_EVENT
[sns_oem1_sensor_instance_island.c  146] Received
SNS_STD_SENSOR_MSGID_SNS_STD_SENSOR_PHYSICAL_CONFIG_EVENT
[sns_oem1_sensor_instance_island.c   49] sns_oem1_inst_process_resampler_event
[sns_oem1_sensor_instance_island.c   39] OEM1-2, timestamp-0x15ce5be85c
[sns_oem1_sensor_instance_island.c  146] Received
SNS_STD_SENSOR_MSGID_SNS_STD_SENSOR_PHYSICAL_CONFIG_EVENT
[sns_oem1_sensor_instance_island.c   49] sns_oem1_inst_process_resampler_event
[sns_oem1_sensor_instance_island.c   39] OEM1-3, timestamp-0x15cfaf0b61
[sns_oem1_sensor_instance_island.c   95] resampler accel data: x 47/1000, y
181/1000, z 9724/1000
[sns_oem1_sensor_instance_island.c  117] OEM1 output:  x 100000/1000, y
181/1000, z 9724/1000
[sns_oem1_sensor_instance_island.c  126] OEM1 last 1 current 0
[sns_oem1_sensor_instance_island.c   49] sns_oem1_inst_process_resampler_event
[sns_oem1_sensor_instance_island.c   39] OEM1-4, timestamp-0x15d0ca7f92
[sns_oem1_sensor_instance_island.c   95] resampler accel data: x 57/1000, y
172/1000, z 9720/1000
```

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 05:57:48 GMT hyungchul.kim@qualcomm.com

```
[sns_oem1_sensor_instance_island.c  117] OEM1 output:  x 100000/1000, y
172/1000, z 9720/1000
```

Sample adb logcat output when OEM1 is streaming (OEM1 is on-change type with resampler enabled):

```
D USTA_Native_Debug: oem1-qualcomm event message count (image (05_24)) is 1
I USTA_APP_INFO: Acquired wakelock using power manager service
I USTA_APP_INFO: Released wakelock using power manager service
D USTA_Native_Debug: is_qmi_disable_api_enabled is false
I USTA_APP_INFO: script Generation is disabled
D USTA_Native_Debug: logFile Name for oem1-qualcomm : is /mnt/sdcard/oem1-
qualcomm_UI_sensorLog.txt
D USTA_Native_Debug: encoded string is empty
D USTA_Native_Debug:  encoded input message for sns_client_request_msg is
generated
D USTA_Native_Debug:
D USTA_Native_Debug: "sns_client_request_msg" : {
D USTA_Native_Debug:  "suid" : {
D USTA_Native_Debug:   "suid_low" : "0xa7167528f835f4ee",
D USTA_Native_Debug:   "suid_high" : "0xe0e3f77b504b96a1"
D USTA_Native_Debug:  },
D USTA_Native_Debug:  "msg_id" : 514,
D USTA_Native_Debug:  "susp_config" : {
D USTA_Native_Debug:   "client_proc_type" : "SNS_STD_CLIENT_PROCESSOR_APSS",
D USTA_Native_Debug:   "delivery_type" : "SNS_CLIENT_DELIVERY_WAKEUP"
D USTA_Native_Debug:  },
D USTA_Native_Debug:  "request" : {
D USTA_Native_Debug:   "batching" : {
D USTA_Native_Debug:    "batch_period" : 0,
D USTA_Native_Debug:    "flush_period" : 0
D USTA_Native_Debug:   }
D USTA_Native_Debug:  }
D USTA_Native_Debug: }
D USTA_Native_Debug:
D USTA_Native_Debug:
D USTA_Native_Debug: "sns_client_response_msg" : {
D USTA_Native_Debug:  "suid" : {
D USTA_Native_Debug:   "suid_low" : "0xa7167528f835f4ee",
D USTA_Native_Debug:   "suid_high" : "0xe0e3f77b504b96a1",
D USTA_Native_Debug: },
D USTA_Native_Debug: "sns_std_error" : {
D USTA_Native_Debug:  "value" : "SNS_STD_ERROR_NO_ERROR"
D USTA_Native_Debug:  }
D USTA_Native_Debug: }
D USTA_Native_Debug: log_message_size 346
D USTA_Native_Debug:
D USTA_Native_Debug: "sns_client_event_msg" : {
D USTA_Native_Debug:  "suid" : {
```

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 07:25:48 GMT hyungchul.won@lge.com

```
D USTA_Native_Debug:   "suid_low" : "0xa7167528f835f4ee",
D USTA_Native_Debug:   "suid_high" : "0xe0e3f77b504b96a1"
D USTA_Native_Debug:  },
D USTA_Native_Debug:  "events" : [
D USTA_Native_Debug:   {
D USTA_Native_Debug:    "msg_id" : 1024,
D USTA_Native_Debug:    "timestamp" : 98664583358,
D USTA_Native_Debug:    "payload" : {
D USTA_Native_Debug:     "oem1" : [
D USTA_Native_Debug:      100.000000,
D USTA_Native_Debug:      0.186740,
D USTA_Native_Debug:      9.720041
D USTA_Native_Debug:     ],
D USTA_Native_Debug:     "accuracy" :
"SNS_STD_SENSOR_SAMPLE_STATUS_ACCURACY_HIGH"
D USTA_Native_Debug:    }
D USTA_Native_Debug:   }
D USTA_Native_Debug:  ]
D USTA_Native_Debug: }
```

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:35:48 GMT
hyungchul.won@lge.com

6 Provide customer algorithms as binary

In some cases, OEMs may want to develop the algorithm and integrate the libraries without the algorithm source code. This can be done by setting the `binary_lib` to `True` in the algorithm .scons file with `AddSSCSU`; this is explained in the following using `sns_oem1.scons` as an example.

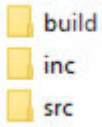
By default, during development, `binary_lib` is set to `False` or "Source" to compile the algorithm source code.

File location is **`slpi_proc\ssc\sensors\oem1\build\sns_oem1.scons`**.

Example:

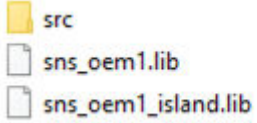
```
if 'USES_SSC_STATIC_LIB_BUILDER' in env:
  env.AddSSCSU(inspect.getfile(inspect.currentframe()),
               register_func_name = "sns_oem1_register",
               clean_pack_list = oem1_clean_pack_list,
               binary_lib =  False,
               add_island_files = oem1_island_enable,
               pb_apis = pb_api,
               diag_ssid = ssid)
```

`slpi_proc\ssc\sensors\oem1` includes these three directories:



`oem1\build` contains the following:

- .lib files in `slpi_proc\ssc\sensors\oem1\build\ssc_slpi_user\qdsp6\<chipset_name>\`
- .o files in `slpi_proc\ssc\sensors\oem1\build\ssc_slpi_user\qdsp6\<chipset_name>\src\`



Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:35:58 GMT
hyungchul.won@lge.com

Delete the following directories:

- .c files – `slpi_proc\ssc\sensors\oem1\src`
- .h files – `slpi_proc\ssc\sensors\oem1\inc`
- .o files – `slpi_proc\ssc\sensors\oem1\build\ssc_slpi_user \qdsp6\<chipset_name>\src`

The following directories remain:

- .lib files in `slpi_proc\ssc\sensors\oem1\build\ssc_slpi_user \qdsp6\<chipset_name>\`
- .scons file in `slpi_proc\ssc\sensors\oem1\build`

Change `binary_lib` to `True` or "Binary" as shown:

File location is **`slpi_proc\ssc\sensors\oem1\build\sns_oem1.scons`**.

Example:

```
    if 'USES_SSC_STATIC_LIB_BUILDER' in env:
env.AddSSCSU(inspect.getfile(inspect.currentframe()),
             register_func_name = "sns_oem1_register",
             clean_pack_list = oem1_clean_pack_list,
             binary_lib =  True,
             add_island_files = oem1_island_enable,
             pb_apis = pb_api,
             diag_ssid = ssid)
```

Compile the build with this change; it will work similar to earlier.

This modified algorithm directory oem1 with .scons files and .lib files can be shipped to end customers for integration on their side.

NOTE If a custom .proto file is added, keep the .proto file as well; it should not be removed.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:35:48 GMT hyungchul.won@lge.com

A References

A.1 Related documents

Title	Number
Qualcomm Technologies, Inc.	
Sensors Execution Environment (SEE) Sensors Deep Dive	80-P9301-35
Sensors Execution Environment Client API Reference	80-P9301-36
Resources	
Google protocol buffers (https://developers.google.com/protocol-buffers/)	
Nanopb (https://jpa.kapsi.fi/nanopb/)	

A.2 Acronyms and terms

Acronym or term	Definition
AAH	All-Ways Aware Hub
DAE	Data acquisition engine
SEE	Sensors Execution Environment
SLPI	Sensors Low Power Island
SSC	Snapdragon Sensors Core
SUID	Sensor unique identifier
USTA	Unified Sensor Test Application

Qualcomm
Confidential – May Contain Trade Secrets
2026-02-27 03:35:48 GMT
hyungchul.kim@qce.com